Universidad Nacional de Colombia

Facultad de Minas

Programación Orientada a Objetos 2026-2

Actividad 3

Interfaces gráficas de usuario y herencia: solución de los ejercicios 8.2 y 8.3

Presentado por:

Natalia Patricia Remolina Rodríguez

Docente:
Walter Hugo Arboleda Mazo

Medellín, Colombia
22 de octubre de 2026

Ejercicio 8.2. Componentes swing: Notas

Enunciado

Se requiere desarrollar un programa con interfaz gráfica de usuario que genere una ventana donde se solicite el ingreso de cinco notas obtenidas por un estudiante. El programa debe calcular y mostrar en la parte inferior de la ventana los siguientes datos:

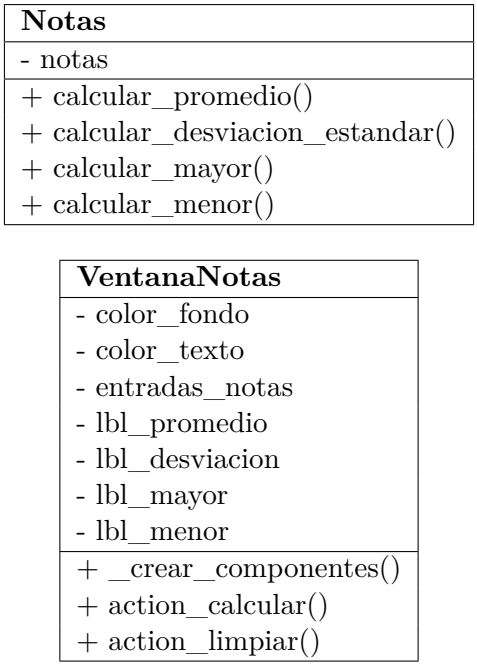
- El promedio de las notas ingresadas.
- La desviación estándar de las notas ingresadas.
- La mayor nota obtenida.
- La menor nota obtenida.

Ejercicios propuestos incluidos en la solución:

- Generar mensajes de alerta cuando se ingresen datos que no sean numéricos.
- Es obligatorio ingresar las cinco notas: se genera un mensaje de alerta cuando alguna nota no ha sido ingresada.

Enlace al repositorio en GitHub:
https://github.com/NatoRemolina/UNAL-Programacion-Orientada-a-Objetos/blob/main/actividad-03/ejercicio1.py

Diagrama de clases



Relaciones:

- `VentanaNotas` hereda de `tk.Tk`: es la ventana principal de la aplicación.
- `VentanaNotas` usa a `Notas` (dependencia): crea un objeto de esa clase cada vez que se presiona el botón *Calcular* y le delega los cálculos. De esta forma la lógica del negocio queda separada de la presentación.

Diagrama de objetos

El siguiente diagrama corresponde al estado de la aplicación en el momento en que se ingresan las notas 5.0, 3.5, 2.5, 4.5 y 3.6 y se presiona el botón *Calcular*.

ventana : VentanaNotas
color_fondo = “#e3f2fd”
color_texto = “#0d47a1”
entradas_notas = [5.0, 3.5, 2.5, 4.5, 3.6]

calculadora : Notas
notas = [5.0, 3.5, 2.5, 4.5, 3.6]

El objeto `ventana` mantiene una referencia temporal a `calculadora` mientras dura el cálculo, y con los valores que este retorna actualiza sus etiquetas: promedio = 3.82, desviación estándar = 0.87, mayor nota = 5.00 y menor nota = 2.50.

Casos de uso

Caso de uso	CU-01: Calcular los indicadores estadísticos
Actor	Usuario (docente o estudiante)
Precondición	La ventana de la calculadora está abierta.
Flujo principal	1. El usuario ingresa las cinco notas. 2. El usuario presiona el botón *Calcular*. 3. El sistema valida que las cinco casillas estén diligenciadas. 4. El sistema valida que todos los valores sean numéricos. 5. El sistema calcula promedio, desviación estándar, mayor y menor. 6. El sistema muestra los cuatro resultados en la parte inferior.
Flujo alterno A	En el paso 3, si alguna casilla está vacía, el sistema muestra la alerta *“Es obligatorio ingresar las cinco notas”* e indica cuáles faltan. El caso de uso termina.
Flujo alterno B	En el paso 4, si algún valor no es numérico, el sistema muestra la alerta *“Por favor, ingrese números válidos”*. El caso de uso termina.
Postcondición	Los cuatro indicadores quedan visibles en la ventana.

Caso de uso	CU-02: Limpiar el formulario
Actor	Usuario
Precondición	La ventana de la calculadora está abierta.
Flujo principal	1. El usuario presiona el botón *Limpiar*. 2. El sistema borra el contenido de las cinco casillas. 3. El sistema restablece las cuatro etiquetas de resultados.
Postcondición	La ventana queda en su estado inicial, lista para un nuevo cálculo.

Interfaz de usuario

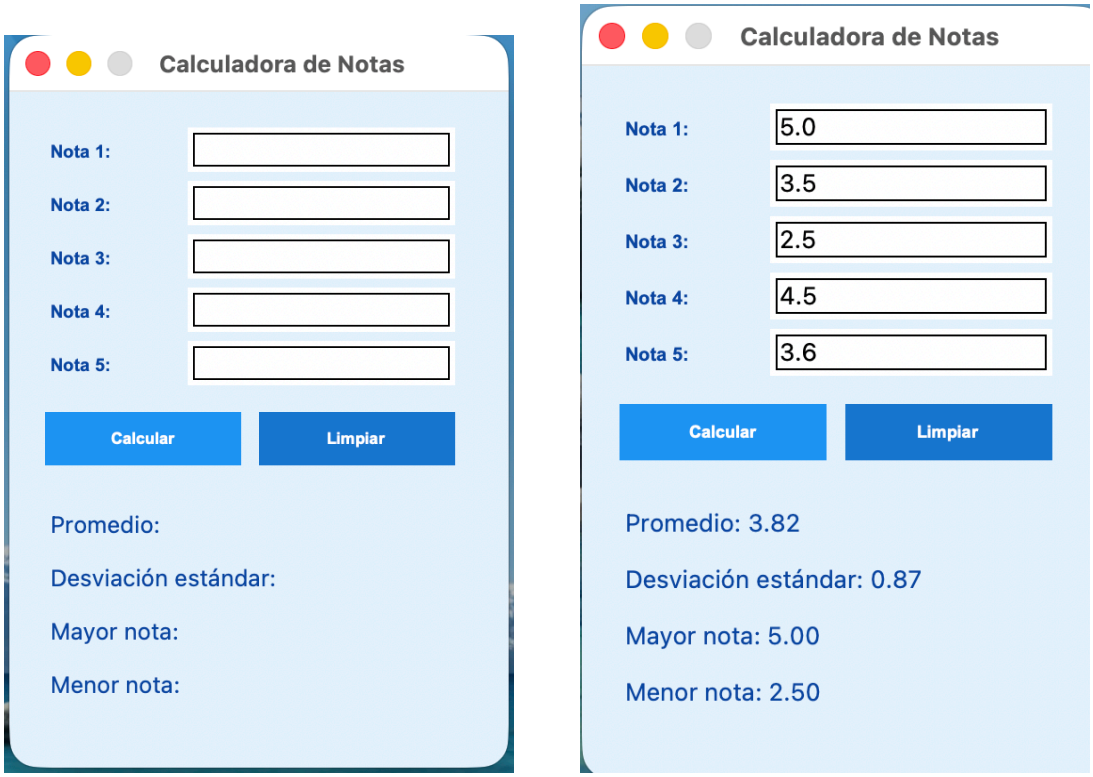


Figura 1: Ventana principal al iniciar la aplicación (izquierda) y resultado del procesamiento de las cinco notas (derecha).

Código fuente

```python
import tkinter as tk
from tkinter import messagebox, ttk
import math

# ==========================================
# CLASE LÓGICA (Modelo)
# ==========================================
class Notas:
    """Clase encargada de realizar los cálculos matemáticos de las notas."""
    def __init__(self, lista_notas):
        self.notas = lista_notas

    def calcular_promedio(self):
        # Retorna el promedio simple de la lista
        return sum(self.notas) / len(self.notas)

    def calcular_desviacion_estandar(self):
        promedio = self.calcular_promedio()
        # Varianza poblacional: suma de (nota - promedio)^2 / N
        varianza = sum(math.pow(nota - promedio, 2) for nota in self.notas) / len(self.notas)
        # Retorna la raíz cuadrada de la varianza
        return math.sqrt(varianza)

```

```
    def calcular_mayor(self):
        # Retorna el valor máximo de la lista
        return max(self.notas)

    def calcular_menor(self):
        # Retorna el valor mínimo de la lista
        return min(self.notas)

# ========================================
# CLASE INTERFAZ GRÁFICA (Vista/Controlador)
# ========================================
class VentanaNotas(tk.Tk):
    """Clase que representa la ventana principal con una paleta de colores azules."""
    def __init__(self):
        super().__init__()

        # Paleta de colores azul
        self.color_fondo = "#e3f2fd" # Azul muy claro
        self.color_texto = "#0d47a1" # Azul oscuro para etiquetas

        # Configuración básica
        self.title("Calculadora de Notas")
        self.geometry("280x380")
        self.resizable(False, False)
        # Aplicamos el color de fondo a toda la ventana
        self.configure(bg=self.color_fondo)

        self.entradas_notas = []

        # Inicializar componentes
        self._crear_componentes()

        self.eval('tk::PlaceWindow . center')

    def _crear_componentes(self):
        # Crear los campos de entrada
        for i in range(5):
            lbl = tk.Label(self, text=f"Nota {i+1}:", anchor="w",
                           bg=self.color_fondo, fg=self.color_texto, font=("Arial", 10, "bold"))
            lbl.place(x=20, y=20 + (i * 30), width=60, height=25)

            txt_entry = tk.Entry(self, bd=1, relief="solid")
            txt_entry.place(x=100, y=20 + (i * 30), width=150, height=25)
            self.entradas_notas.append(txt_entry)

        # Botones con estilo azul.
        # Se usa ttk con el tema "clam" porque los botones tk no admiten
        # colores de fondo en macOS y quedarían ilegibles.
        estilo = ttk.Style(self)
        estilo.theme_use("clam")
        estilo.configure("Calcular.TButton", background="#2196f3", foreground="white",
                         font=("Arial", 9, "bold"), borderwidth=0, focuscolor="#2196f3")
        estilo.map("Calcular.TButton", background=[("active", "#1565c0")
```

```
    ])
77          estilo.configure("Limpiar.TButton", background="#1976d2",
    foreground="white",
78                           font=("Arial", 9, "bold"), borderwidth=0,
    focuscolor="#1976d2")
79          estilo.map("Limpiar.TButton", background=[("active", "#0d47a1")
    ])
80
81          btn_calcular = ttk.Button(self, text="Calcular", command=self.
    action_calcular,
82                                    style="Calcular.TButton")
83          btn_calcular.place(x=20, y=180, width=110, height=30)
84
85          btn_limpiar = ttk.Button(self, text="Limpiar", command=self.
    action_limpiar,
86                                   style="Limpiar.TButton")
87          btn_limpiar.place(x=140, y=180, width=110, height=30)
88
89          # Labels de resultados con estilo azul
90          resultado_style = {"bg": self.color_fondo, "fg": self.
    color_texto, "anchor": "w"}
91
92          self.lbl_promedio = tk.Label(self, text="Promedio: ", **
    resultado_style)
93          self.lbl_promedio.place(x=20, y=230, width=240, height=25)
94
95          self.lbl_desviacion = tk.Label(self, text="Desviación estándar:
    ", **resultado_style)
96          self.lbl_desviacion.place(x=20, y=260, width=240, height=25)
97
98          self.lbl_mayor = tk.Label(self, text="Mayor nota: ", **
    resultado_style)
99          self.lbl_mayor.place(x=20, y=290, width=240, height=25)
100
101         self.lbl_menor = tk.Label(self, text="Menor nota: ", **
    resultado_style)
102         self.lbl_menor.place(x=20, y=320, width=240, height=25)
103
104     def action_calcular(self):
105         """Procesa las entradas y muestra resultados."""
106         # Es obligatorio ingresar las cinco notas: se avisa cuáles
    faltan
107         faltantes = [i + 1 for i, entry in enumerate(self.entradas_notas
    )
108                      if not entry.get().strip()]
109         if faltantes:
110             notas_faltantes = ", ".join(str(numero) for numero in
    faltantes)
111             messagebox.showwarning(
112                 "Datos incompletos",
113                 f"Es obligatorio ingresar las cinco notas.\nFalta la
    nota: {notas_faltantes}."
114             )
115             return
116
117         try:
118             valores = [float(entry.get()) for entry in self.
    entradas_notas]
```

```
119             calculadora = Notas(valores)
120 
121             # Actualizamos textos con los cálculos
122             self.lbl_promedio.config(text=f"Promedio: {calculadora.
    calcular_promedio():.2f}")
123             self.lbl_desviacion.config(text=f"Desviación estándar: {
    calculadora.calcular_desviacion_estandar():.2f}")
124             self.lbl_mayor.config(text=f"Mayor nota: {calculadora.
    calcular_mayor():.2f}")
125             self.lbl_menor.config(text=f"Menor nota: {calculadora.
    calcular_menor():.2f}")
126 
127         except ValueError:
128             messagebox.showerror("Error", "Por favor, ingrese números vá
    lidos.")
129 
130     def action_limpiar(self):
131         """Limpia la interfaz."""
132         for entry in self.entradas_notas:
133             entry.delete(0, tk.END)
134         self.lbl_promedio.config(text="Promedio: ")
135         self.lbl_desviacion.config(text="Desviación estándar: ")
136         self.lbl_mayor.config(text="Mayor nota: ")
137         self.lbl_menor.config(text="Menor nota: ")
138 
139 # ==========================================
140 # BLOQUE PRINCIPAL
141 # ==========================================
142 if __name__ == "__main__":
143     app = VentanaNotas()
144     app.mainloop()
```

Ejercicio 8.3. Gestión de eventos: figuras geométricas

Enunciado

Se requiere desarrollar un programa con interfaz gráfica de usuario que permita calcular el volumen y la superficie de varias figuras geométricas:

- Para el cilindro se solicitan su radio y altura (en centímetros).
- Para la esfera, su radio (en centímetros).
- Para la pirámide, su base, altura y apotema (en centímetros).

Para desarrollar el programa se debe crear una jerarquía de clases para las diferentes figuras geométricas requeridas.
Ejercicios propuestos incluidos en la solución:

- Agregar el cálculo del volumen y la superficie de otras figuras geométricas como el cubo y el prisma.
- Agregar a cada figura geométrica su imagen correspondiente.

Enlace al repositorio en GitHub:
https://github.com/NatoRemolina/UNAL-Programacion-Orientada-a-Objetos/blob/main/actividad-03/ejercicio2.py

Diagrama de clases

Jerarquía de figuras geométricas:

FiguraGeometrica
- volumen - superficie
+ get_volumen() + get_superficie()

Cilindro
- radio - altura
+ calcular_volumen() + calcular_superficie()

Esfera
- radio
+ calcular_volumen() + calcular_superficie()

Piramide
- base - altura - apotema
+ calcular_volumen() + calcular_superficie()

Cubo
- lado
+ calcular_volumen() + calcular_superficie()

Prisma
- largo - ancho - altura
+ calcular_volumen() + calcular_superficie()

Jerarquía de ventanas:

VentanaPrincipal
(sin atributos propios)
+ abrir_cilindro() + abrir_esfera() + abrir_piramide() + abrir_cubo() + abrir_prisma()

VentanaFigura
- entradas - lbl_volumen - lbl_superficie
+ crear_figura() + calcular()

Relaciones:

- `Cilindro`, `Esfera`, `Piramide`, `Cubo` y `Prisma` heredan de `FiguraGeometrica`, que concentra los atributos y los métodos de consulta comunes a todas las figuras.
- `VentanaPrincipal` hereda de `tk.Tk` y es la ventana del menú: abre la ventana correspondiente a cada figura.
- `VentanaFigura` hereda de `tk.Toplevel` y define la ventana común a todas las figuras: la imagen, los campos de entrada, el botón *Calcular* y la presentación de los resultados.
- `VentanaCilindro`, `VentanaEsfera`, `VentanaPiramide`, `VentanaCubo` y `VentanaPrisma` heredan de `VentanaFigura` y únicamente redefinen el método `crear_figura()`, que construye el objeto de la figura con los valores ingresados.

Diagrama de objetos

El siguiente diagrama corresponde al estado del programa después de calcular el volumen y la superficie de cada una de las cinco figuras con los valores de ejemplo.

cil : Cilindro
radio = 2.0 altura = 5.0 volumen = 62.83 superficie = 87.96

esf : Esfera
radio = 3.0 volumen = 113.10 superficie = 113.10

pir : Piramide
base = 4.0 altura = 6.0 apotema = 5.0 volumen = 32.00 superficie = 56.00

cub : Cubo
lado = 3.0
volumen = 27.00
superficie = 54.00

pri : Prisma
largo = 2.0
ancho = 3.0
altura = 4.0
volumen = 24.00
superficie = 52.00

Cada uno de estos objetos es creado por su ventana correspondiente (`ventana_cilindro : VentanaCilindro`, `ventana_esfera : VentanaEsfera`, y así sucesivamente), las cuales a su vez son abiertas por el objeto `menu : VentanaPrincipal`.

Casos de uso

Caso de uso	CU-01: Seleccionar una figura geométrica
Actor	Usuario
Precondición	El programa está en ejecución y el menú principal está visible.
Flujo principal	1. El usuario presiona el botón de una de las cinco figuras. 2. El sistema abre la ventana de esa figura. 3. El sistema muestra la imagen de la figura y los campos que corresponden a sus dimensiones.
Postcondición	La ventana de la figura queda abierta y lista para recibir datos.

Caso de uso	CU-02: Calcular volumen y superficie
Actor	Usuario
Precondición	La ventana de la figura seleccionada está abierta (CU-01).
Flujo principal	1. El usuario ingresa las dimensiones solicitadas en centímetros. 2. El usuario presiona el botón *Calcular*. 3. El sistema convierte los valores ingresados a números. 4. El sistema crea el objeto de la figura correspondiente. 5. El objeto calcula su volumen y su superficie. 6. El sistema muestra ambos resultados con dos decimales.
Flujo alterno A	En el paso 3, si algún valor no es numérico, el sistema muestra la alerta *"Ingrese valores numéricos válidos"* y no realiza el cálculo. El caso de uso termina.
Postcondición	El volumen y la superficie quedan visibles en la ventana.

Interfaz de usuario

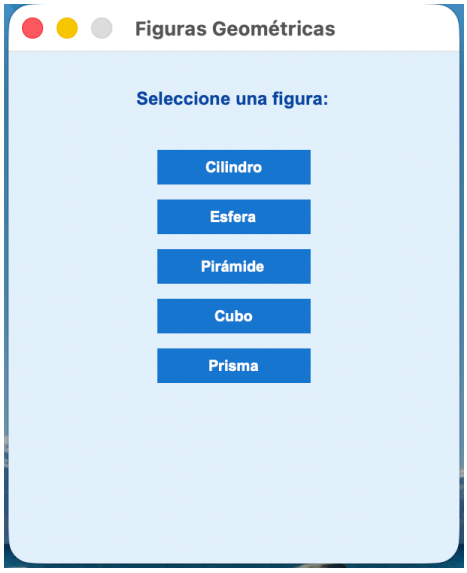


Figura 2: Menú principal con las cinco figuras geométricas disponibles.

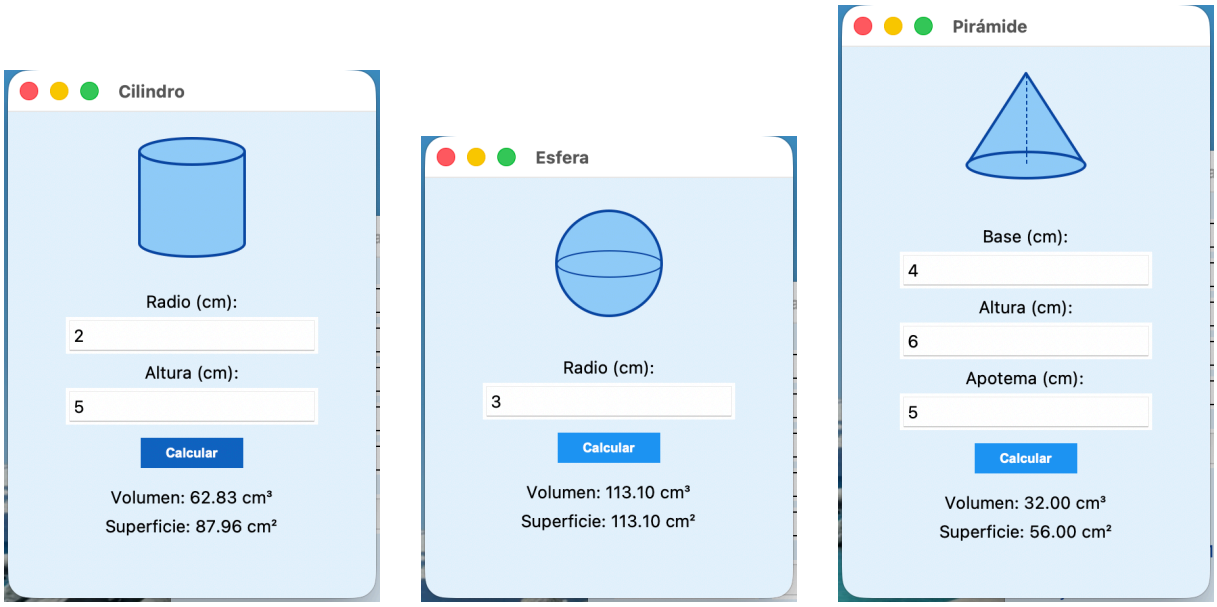


Figura 3: Ventanas del cilindro, la esfera y la pirámide, cada una con su imagen y sus resultados.

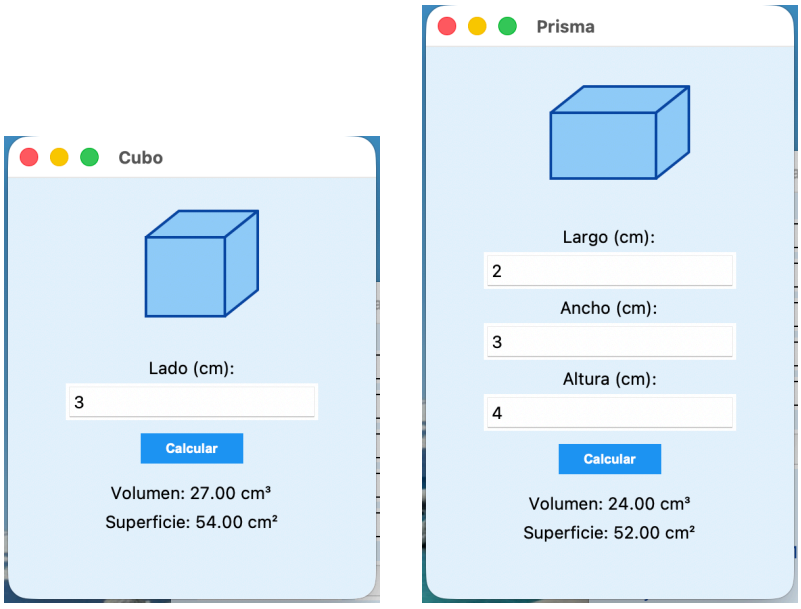


Figura 4: Ventanas del cubo y del prisma, figuras agregadas como parte de los ejercicios propuestos.

Código fuente

```
import tkinter as tk
from tkinter import messagebox, ttk
import math

# ==========================================
# JERARQUÍA DE CLASES (Lógica del Negocio)
# ==========================================

class FiguraGeometrica:
    """Clase base que almacena los resultados de los cálculos."""
    def __init__(self):
```

```
        self.volumen = 0.0
        self.superficie = 0.0

    def get_volumen(self):
        return self.volumen

    def get_superficie(self):
        return self.superficie

class Cilindro(FiguraGeometrica):
    def __init__(self, radio, altura):
        super().__init__()
        self.radio = radio
        self.altura = altura
        self.calcular_volumen()
        self.calcular_superficie()

    def calcular_volumen(self):
        # Fórmula: pi * r^2 * h
        self.volumen = math.pi * (self.radio ** 2) * self.altura

    def calcular_superficie(self):
        # Fórmula: 2 * pi * r * h + 2 * pi * r^2
        self.superficie = (2 * math.pi * self.radio * self.altura) + (2
   * math.pi * (self.radio ** 2))

class Esfera(FiguraGeometrica):
    def __init__(self, radio):
        super().__init__()
        self.radio = radio
        self.calcular_volumen()
        self.calcular_superficie()

    def calcular_volumen(self):
        # Fórmula: 4/3 * pi * r^3
        self.volumen = (4.0 / 3.0) * math.pi * (self.radio ** 3)

    def calcular_superficie(self):
        # Fórmula: 4 * pi * r^2
        self.superficie = 4.0 * math.pi * (self.radio ** 2)

class Piramide(FiguraGeometrica):
    def __init__(self, base, altura, apotema):
        super().__init__()
        self.base = base
        self.altura = altura
        self.apotema = apotema
        self.calcular_volumen()
        self.calcular_superficie()

    def calcular_volumen(self):
        # Fórmula: (base^2 * altura) / 3
        self.volumen = ((self.base ** 2) * self.altura) / 3.0

    def calcular_superficie(self):
        # Fórmula: base^2 + 2 * base * apotema
        self.superficie = (self.base ** 2) + (2.0 * self.base * self.
   apotema)
```

```
68
69  class Cubo(FiguraGeometrica):
70      def __init__(self, lado):
71          super().__init__()
72          self.lado = lado
73          self.calcular_volumen()
74          self.calcular_superficie()
75
76      def calcular_volumen(self):
77          # Fórmula: lado^3
78          self.volumen = self.lado ** 3
79
80      def calcular_superficie(self):
81          # Fórmula: 6 * lado^2
82          self.superficie = 6.0 * (self.lado ** 2)
83
84  class Prisma(FiguraGeometrica):
85      """Prisma rectangular definido por su largo, ancho y altura."""
86      def __init__(self, largo, ancho, altura):
87          super().__init__()
88          self.largo = largo
89          self.ancho = ancho
90          self.altura = altura
91          self.calcular_volumen()
92          self.calcular_superficie()
93
94      def calcular_volumen(self):
95          # Fórmula: largo * ancho * altura
96          self.volumen = self.largo * self.ancho * self.altura
97
98      def calcular_superficie(self):
99          # Fórmula: 2 * (largo*ancho + largo*altura + ancho*altura)
100         self.superficie = 2.0 * ((self.largo * self.ancho) +
101                                  (self.largo * self.altura) +
102                                  (self.ancho * self.altura))
103
104
105 # ========================================
106 # IMÁGENES DE LAS FIGURAS (dibujadas con Canvas)
107 # ========================================
108
109 COLOR_FONDO = "#e3f2fd"
110 COLOR_BORDE = "#0d47a1"
111 COLOR_RELLENO = "#90caf9"
112
113 def configurar_estilos(ventana):
114     """Define el estilo azul de los botones.
115
116     Se usa ttk con el tema "clam" porque los botones tk no admiten
    colores
117     de fondo en macOS y el texto blanco quedaría ilegible.
118     """
119     estilo = ttk.Style(ventana)
120     estilo.theme_use("clam")
121     estilo.configure("Menu.TButton", background="#1976d2", foreground="
    white",
122                      font=("Arial", 10, "bold"), borderwidth=0,
    focuscolor="#1976d2")
```

```
123     estilo.map("Menu.TButton", background=[("active", "#0d47a1")])
124     estilo.configure("Calcular.TButton", background="#2196f3",
    foreground="white",
125                      font=("Arial", 10, "bold"), borderwidth=0,
    focuscolor="#2196f3")
126     estilo.map("Calcular.TButton", background=[("active", "#1565c0")])
127 
128 def dibujar_figura(canvas, tipo):
129     """Dibuja sobre el canvas la imagen de la figura geométrica indicada
    ."""
130     estilo = {"outline": COLOR_BORDE, "fill": COLOR_RELLENO, "width": 2}
131 
132     if tipo == "cilindro":
133         canvas.create_oval(40, 85, 120, 105, **estilo)
134         canvas.create_rectangle(40, 25, 120, 95, outline="", fill=
    COLOR_RELLENO)
135         canvas.create_line(40, 25, 40, 95, fill=COLOR_BORDE, width=2)
136         canvas.create_line(120, 25, 120, 95, fill=COLOR_BORDE, width=2)
137         canvas.create_oval(40, 15, 120, 35, **estilo)
138 
139     elif tipo == "esfera":
140         canvas.create_oval(40, 20, 120, 100, **estilo)
141         canvas.create_oval(40, 50, 120, 70, outline=COLOR_BORDE, width
    =1)
142 
143     elif tipo == "piramide":
144         canvas.create_polygon(80, 15, 35, 85, 125, 85,
145                               outline=COLOR_BORDE, fill=COLOR_RELLENO,
    width=2)
146         canvas.create_oval(35, 75, 125, 95, outline=COLOR_BORDE, fill=
    COLOR_RELLENO, width=2)
147         canvas.create_line(80, 15, 80, 85, fill=COLOR_BORDE, width=1,
    dash=(3, 2))
148 
149     elif tipo == "cubo":
150         canvas.create_polygon(45, 40, 70, 20, 130, 20, 105, 40,
151                               outline=COLOR_BORDE, fill=COLOR_RELLENO,
    width=2)
152         canvas.create_polygon(105, 40, 130, 20, 130, 80, 105, 100,
153                               outline=COLOR_BORDE, fill=COLOR_RELLENO,
    width=2)
154         canvas.create_rectangle(45, 40, 105, 100, **estilo)
155 
156     elif tipo == "prisma":
157         canvas.create_polygon(35, 45, 60, 25, 140, 25, 115, 45,
158                               outline=COLOR_BORDE, fill=COLOR_RELLENO,
    width=2)
159         canvas.create_polygon(115, 45, 140, 25, 140, 75, 115, 95,
160                               outline=COLOR_BORDE, fill=COLOR_RELLENO,
    width=2)
161         canvas.create_rectangle(35, 45, 115, 95, **estilo)
162 
163 
164 # ========================================
165 # INTERFAZ GRÁFICA (Tkinter)
166 # ========================================
167 
168 class VentanaPrincipal(tk.Tk):
```

```
169     def __init__(self):
170         super().__init__()
171         self.title("Figuras Geométricas")
172         self.geometry("300x340")
173         self.configure(bg=COLOR_FONDO) # Fondo azul claro
174         self.resizable(False, False)
175         self.eval('tk::PlaceWindow . center')
176 
177         # Estilo para botones
178         configurar_estilos(self)
179         btn_style = {"width": 15, "style": "Menu.TButton"}
180 
181         tk.Label(self, text="Seleccione una figura:", font=("Arial", 12,
    "bold"),
182                  bg=COLOR_FONDO, fg=COLOR_BORDE).pack(pady=20)
183 
184         ttk.Button(self, text="Cilindro", command=self.abrir_cilindro,
    **btn_style).pack(pady=5)
185         ttk.Button(self, text="Esfera", command=self.abrir_esfera, **
    btn_style).pack(pady=5)
186         ttk.Button(self, text="Pirámide", command=self.abrir_piramide,
    **btn_style).pack(pady=5)
187         ttk.Button(self, text="Cubo", command=self.abrir_cubo, **
    btn_style).pack(pady=5)
188         ttk.Button(self, text="Prisma", command=self.abrir_prisma, **
    btn_style).pack(pady=5)
189 
190     def abrir_cilindro(self): VentanaCilindro(self)
191     def abrir_esfera(self): VentanaEsfera(self)
192     def abrir_piramide(self): VentanaPiramide(self)
193     def abrir_cubo(self): VentanaCubo(self)
194     def abrir_prisma(self): VentanaPrisma(self)
195 
196 class VentanaFigura(tk.Toplevel):
197     """Ventana base: muestra la imagen de la figura y sus campos de
    entrada."""
198     def __init__(self, parent, titulo, tipo, etiquetas, alto):
199         super().__init__(parent)
200         self.title(titulo)
201         self.geometry(f"280x{alto}")
202         self.configure(bg=COLOR_FONDO)
203 
204         # Imagen de la figura geométrica
205         canvas = tk.Canvas(self, width=160, height=120, bg=COLOR_FONDO,
    highlightthickness=0)
206         canvas.pack(pady=5)
207         dibujar_figura(canvas, tipo)
208 
209         # Un campo de entrada por cada dato solicitado
210         self.entradas = []
211         for etiqueta in etiquetas:
212             tk.Label(self, text=etiqueta, bg=COLOR_FONDO).pack(pady=2)
213             txt_entry = tk.Entry(self)
214             txt_entry.pack()
215             self.entradas.append(txt_entry)
216 
217         ttk.Button(self, text="Calcular", command=self.calcular,
218                    style="Calcular.TButton").pack(pady=10)
```

```
219
220         self.lbl_volumen = tk.Label(self, text="Volumen: ", bg=
    COLOR_FONDO)
221         self.lbl_volumen.pack()
222         self.lbl_superficie = tk.Label(self, text="Superficie: ", bg=
    COLOR_FONDO)
223         self.lbl_superficie.pack()
224
225     def crear_figura(self, valores):
226         """Cada ventana construye su propia figura con los valores
    ingresados."""
227         raise NotImplementedError
228
229     def calcular(self):
230         try:
231             valores = [float(entrada.get()) for entrada in self.entradas
    ]
232             figura = self.crear_figura(valores)
233             self.lbl_volumen.config(text=f"Volumen: {figura.get_volumen
    ():.2f} cm³")
234             self.lbl_superficie.config(text=f"Superficie: {figura.
    get_superficie():.2f} cm²")
235         except ValueError:
236             messagebox.showerror("Error", "Ingrese valores numéricos vá
    lidos.")
237
238 class VentanaCilindro(VentanaFigura):
239     def __init__(self, parent):
240         super().__init__(parent, "Cilindro", "cilindro",
241                          ["Radio (cm):", "Altura (cm):"], 370)
242
243     def crear_figura(self, valores):
244         return Cilindro(valores[0], valores[1])
245
246 class VentanaEsfera(VentanaFigura):
247     def __init__(self, parent):
248         super().__init__(parent, "Esfera", "esfera",
249                          ["Radio (cm):"], 320)
250
251     def crear_figura(self, valores):
252         return Esfera(valores[0])
253
254 class VentanaPiramide(VentanaFigura):
255     def __init__(self, parent):
256         super().__init__(parent, "Pirámide", "piramide",
257                          ["Base (cm):", "Altura (cm):", "Apotema (cm):"
    ], 420)
258
259     def crear_figura(self, valores):
260         return Piramide(valores[0], valores[1], valores[2])
261
262 class VentanaCubo(VentanaFigura):
263     def __init__(self, parent):
264         super().__init__(parent, "Cubo", "cubo",
265                          ["Lado (cm):"], 320)
266
267     def crear_figura(self, valores):
268         return Cubo(valores[0])
```

```
269
270 class VentanaPrisma(VentanaFigura):
271     def __init__(self, parent):
272         super().__init__(parent, "Prisma", "prisma",
273                          ["Largo (cm):", "Ancho (cm):", "Altura (cm):"],
    420)
274
275     def crear_figura(self, valores):
276         return Prisma(valores[0], valores[1], valores[2])
277
278 if __name__ == "__main__":
279     app = VentanaPrincipal()
280     app.mainloop()
```